

LOB Project Guide - what you're building and why

A limit order book / matching engine in C++: the piece of exchange infrastructure that sits between "order arrives" and "trade happens." This is a literal interview question at Jane Street, Optiver, Citadel, and HRT - building it is interview prep by construction.

Division of labor

What the AI built (the harness - plumbing, not the portfolio piece):

- `NaiveBook` - a deliberately simple, correct baseline (std::map + deque + hash map). It exists so every test and benchmark runs today, and so you have a number to beat.
- Replay pipeline - CSV event format, `read\_events` / `run`, `lob\_replay` CLI.
- Data generator - deterministic synthetic order flow (`lob\_gen`).
- Benchmark harness - per-event latency at p50/p90/p99/p99.9 + throughput.
- Test suite - 9 behavioral tests + 4 property tests, all running on NaiveBook now; one macro flip runs them on your Book.

What YOU own (the thing interviewers ask about):

- `Book` (`include/lob/book.hpp`, `src/book.cpp`) - the fast implementation. Same observable behavior as NaiveBook, engineered for latency. Every method currently throws - that's the starting line.

The rule, same as CHIP-8: the harness tells you IF you're right and how fast you are. The book is where the learning and the resume lines live.

How the pieces fit

```

event stream (CSV / generated)
  |
  v
+-----+      trades out      +-----+
| Book   | -----> | replay / bench / |
| (yours) | <--- add/cancel/mod | tests record them|
+-----+      +-----+
  ^
  | queries: best_bid, best_ask, order_count, resting_qty
  | tests + property checks (never crossed, FIFO, conservation)

```

Your Book and NaiveBook share one contract. Tests don't care which they run against - that's the point: correctness is defined by behavior, then you optimize inside the contract.

The contract (what your Book must do)

1. Price-time priority: best price matches first; ties break FIFO.

2. Limit orders match while they cross; leftover rests only if GTC.
3. Market orders take whatever is available; never rest.
4. IOC fills what it can, drops the rest. FOK fills fully or not at all.
5. ``cancel(id)`` -> bool. ``modify(id, px, qty)`` = cancel + re-add, so it loses queue priority. Unknown ids return false.
6. Trades report at the RESTING order's price.

All of this is already testable - ``naive__*`` tests encode it.

Milestones (roughly in order)

M0 - Get oriented (day 1)

- Run ``build.bat test``, ``build/lob_gen``, ``lob_replay``, ``lob_bench``.
- Read ``include/lob/types.hpp`` and ``naive_book.hpp`` until you can explain every field. Sketch the lifecycle of one order on paper.
- Learn: what price-time priority means, what "crossing the spread" means, why trades print at the resting price.

M1 - Working Book, naive internals (day 1-2)

- Implement ``src/book.cpp`` however is easiest - copying the `std::map` approach is FINE. Goal: ``BOOK_SUITE(fast_, lob::Book)`` uncommented and all tests pass. Now you're green; optimization is pure fun.
- Learn: the value of "correct first, fast second" - you now have a differential-testing setup (naive vs fast) for free.

M2 - The O(1) redesign (days 3-6, the core work)

- Replace ``std::map<Price, deque<Order>>`` with your own structure:
 - Order objects in a pool/arena (no per-order new/delete).
 - Intrusive doubly-linked FIFO per level (orders link to each other; cancel = unlink two pointers = O(1), no search).
 - ``unordered_map<OrderId, Order*>`` for O(1) cancel lookup.
 - Price levels: `std::map` is fine at first; the stretch is a flat array indexed by (price - lo)/tick when the instrument's range is bounded, or an order-list + bitmap.
- Re-run tests after EACH structural change. Property tests catch subtle priority bugs that unit tests miss.
- Learn: intrusive data structures, cache locality, why deque/pointer chasing kills latency, allocator design, O(1) vs O(log n) in practice (constants matter more than big-O at this scale).

M3 - Measure like a professional (days 7-8)

- ``lob_bench --gen 1000000 --book fast`` vs ``--book naive``.
- Report percentiles (p50/p99/p99.9), never just the mean - tail latency is the whole story in this domain.

- Try: different seeds, event mixes (more cancels), book depths.
- Learn: how to read a latency distribution, what a benchmark must control for (warmup, allocator state, data layout), honest reporting - "my fast version was SLOWER at X, here's why" earns more credibility than a headline number.

M4 - The writeup (days 9-10)

- Benchmark table: naive vs fast, per event kind, percentile columns.
- One paragraph per design decision: what you chose, what you rejected, what surprised you.
- This doc + numbers IS the demo. Post it with the repo.

Stretch (pick one if time remains)

- NASDAQ ITCH-5.0 replay: parse real total-view itch data into your Event stream. This is what got the LinkedIn posts traction.
- Arena allocator for Orders (if not done in M2).
- Depth-chart visualization (even ASCII top-10-levels dump per second).
- Modify semantics variants: qty-decrease-only keeps priority (some venues do this - great interview tangent).

What to learn (the actual syllabus)

- Data structures: intrusive linked lists, object pools/arenas, hash maps, sorted containers, flat arrays vs trees.
- Systems: cache lines & locality, pointer chasing costs, branch prediction, allocation in hot paths, why `std::function/virtual` are avoided in low-latency code.
- Domain: order types (GTC/IOC/FOK), matching semantics, queue priority, venue differences, what a feed handler does.
- Method: property-based thinking (invariants that must ALWAYS hold), differential testing (two impls, same input, same output), deterministic replay for debugging, percentile latency reporting.

Interview questions this prepares you for

- "Design an order book." (You'll have done it, with numbers.)
- "How do you make cancel $O(1)$?" (`OrderId` -> `Order*` + intrusive list.)
- "Why is p99.9 worse than p50?" (Allocator, cache misses, level creation/teardown - measure it, then say so.)
- "How do you know it's correct?" (Property tests + differential testing vs a reference implementation + deterministic replay.)
- "Walk me through a trade." (Aggressor crosses, FIFO at level, partial fills, trade at resting price, residual rests or dies.)

Commands cheat sheet

```
build.bat                build everything
build.bat test           build + run all tests
build/lob_gen --out d.csv --orders 1000000 --seed 42
build/lob_replay d.csv --trades      replay, print every trade
build/lob_bench --gen 1000000 --book naive --runs 5
build/lob_bench --gen 1000000 --book fast  --runs 5
```

Enable your Book in tests: uncomment `BOOK\_SUITE(fast\_, lob::Book)`
in tests/test_book.cpp.

Regenerating this PDF

```
python tools/make_guide.py    -> GUIDE.pdf
```